

```
int *pa, *pb, *pc;
/* is same as: */
int *pa;
int *pb;
int *pc;
```

Once declared, a pointer can usually be reassigned so that it points to an object of another type. CCS lets you reassign pointers without typecasting, but the compiler will warn you unless the pointer was originally declared to be pointing to void. You can assign a void* pointer to a non-void* pointer – refer to void for details.

Null pointers

A null pointer value is an address that is guaranteed to be different from any valid pointer in use in a program. Assigning the integer constant 0 to a pointer assigns a null pointer value to it. Instead of zero, the mnemonic NULL (defined in the standard library header files, such as stdio.h) can be used for legibility. All pointers can be successfully tested for equality or inequality to NULL.

For example:

```
int *pn = 0;          /* Here's one null pointer */
int *pn = NULL;       /* This is an equivalent declaration */

/* We can test the pointer like this: */
if ( pn == 0 ) { ... }

/* .. or like this: */
if ( pn == NULL ) { ... }
```

The pointer type “pointer to void” must not be confused with the null pointer. The declaration `void *vp;` declares that `vp` is a generic pointer capable of being assigned to by any “pointer to type” value, including null, without complaint.

Assignments without proper casting between a “pointer to type1” and a “pointer to type2”, where type1 and type2 are different types, can invoke a compiler warning or error. If type1 is a function and type2 isn’t (or vice versa), pointer assignments are illegal. If type1 is a pointer to void, no cast is needed. If type2 is a pointer to void, no cast is needed.